Multi-Agent Systems

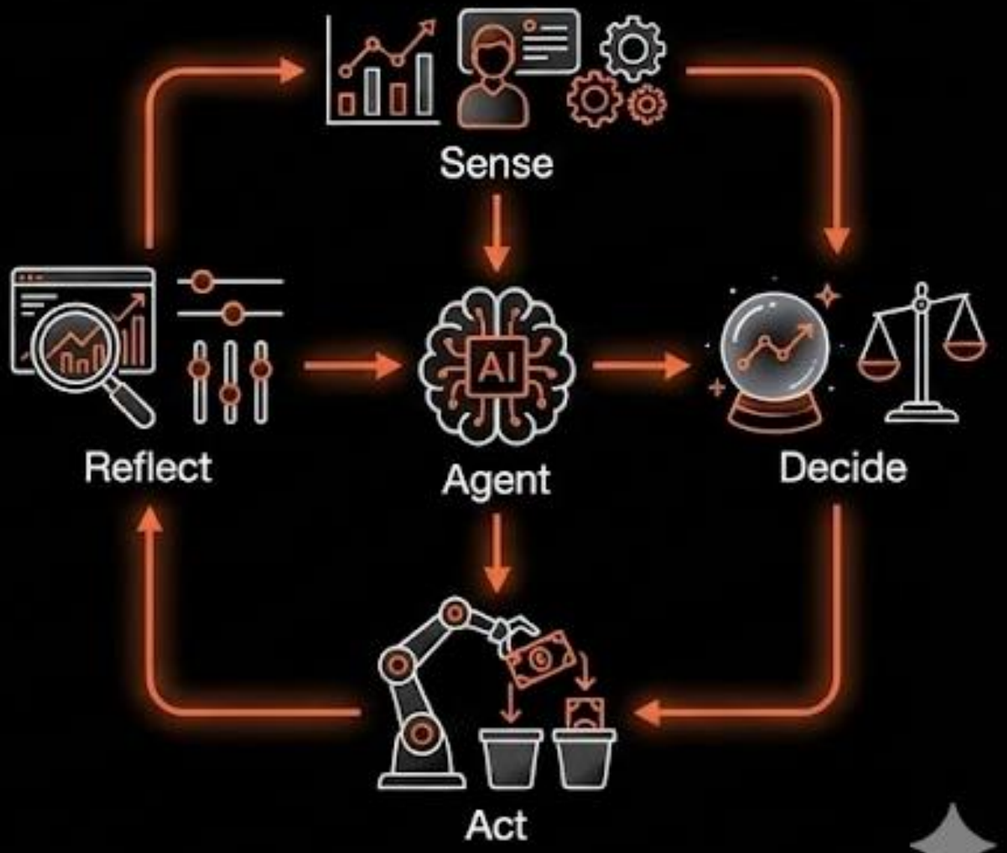
From mathematical models to generative AI

Andy Li, PhD, CFA, FRM

Agentic Portfolio Rebalancing in Action

Feedback loop: Sense → Decide → Act → Reflect

- Agents observe: market data, investor profiles, constraints
- Plan: forecast market regime, assess drift
- Act: execute rebalance with cost/risk-awareness
- Reflect: review effect of action, adjust threshold



Agentic Finance: Rebalancing & Governance

The Agent's Feedback Loop & Safeguards



OBSERVE:
Market Data, Investor Profiles, Constraints



PLAN:
Forecast Regime, Assess Drift





EVALUATE:
Review Effects, Adjust Thresholds



EXECUTE:
Rebalance with Cost/Risk Awareness

Performance Metrics: Sharpe, Sortino, Drawdown, Tracking Error, Turnover

Ensures: Trust, Explainability, Regulatory Alignment

AI Agent Trends

Trend 1: Stronger Reasoning Ability

```
Current (2024):
  GPT-4: Good at language understanding and generation
  Limited reasoning ability (complex math, logic)

Future (2025-2026):
  OpenAI o1 series: Focused on reasoning
  DeepSeek-R1: Reinforcement learning reasoning
  Reasoning time↑, Reasoning accuracy↑

Impact:
  Agent can handle more complex tasks
  Reduced reliance on external tools
  Fewer errors and hallucinations
```

Trend 2: Multimodal Agent

```
Current: Mainly processes text

Future:
  Image understanding (recognizing charts, UI design)
  Video analysis (editing, content moderation)
  Voice interaction (real-time dialogue)
  3D environment operation (gaming, simulation)
Example Scenario: User uploads design sketch → Agent
recognition → Generate HTML/CSS → Automatic website
deployment
```

Trend 3: Personalization and Memory Enhancement

```
Current: Dialogue-level memory
Future:
  Cross-session long-term memory
  Personalized learning (understanding user habits)
  Proactive suggestions (not waiting for user query)
```



Why Multi-Agent Systems in Finance?

Financial markets are **distributed by design**: thousands of actors, diverse objectives. Multi-Agent Systems (MAS) mirror reality:

- Teams of specialists
- Parallelism and redundancy
- Modular, upgradable components

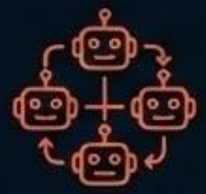
From Single Agent → Multi-Agent Ecosystem



Single Agent

One LLM controlling reasoning, acting, tool use.

- ✓ Pros: simple, less coordination overhead.
- ✗ Cons: rigid, brittle, lacks specialization.



Multi-Agent System (MAS)

Multiple agents with distinct roles.
Communicate, negotiate, and delegate.

- ✓ Pros: specialization, scalability, fault tolerance.
- ✗ Cons: harder to implement

FINANCIAL USE CASE LANDSCAPE



Trading & Execution: Signal discovery, order execution, risk checks → different agents.



Portfolio Optimization: Allocation, hedging, compliance → distributed tasks.



Fraud Detection: Anomaly agents monitor patterns.



Compliance & Audit: Rule-checking agents, real-time reporting.



Market Simulation: Swarm of trader agents testing scenarios.

WHY SPLIT TASKS?

- **Specialization**: each agent is optimized for a narrow function. Example: risk agent only focuses on drawdown, not execution speed.
- **Parallelism**: tasks run concurrently (execution + compliance).
- **Robustness**: failure of one agent doesn't collapse the whole system.
- **Transparency**: easier to audit sub-agents with defined scopes.
- **Modularity**: agents can be swapped/upgraded independently.

Example: Portfolio Management MAS



Example: Fraud Detection MAS



Example: Compliance MAS

- Trade agent proposes order.
- Compliance agent cross-checks policies.
- Audit agent logs justification.
- Human-in-the-loop checkpoint for final approval.

Market Simulation MAS

- Swarm of agents simulating different trader profiles:
  - Momentum traders
  - Value investors
  - HFT arbitrage agents



- Emergent behavior → flash crashes, liquidity droughts.
- **Useful for**: stress tests, regulatory sandboxing.

MAS Advantages in Finance

Core Advantages



Scalable problem solving

Multi-agent systems break complex tasks into smaller, parallel workflows.



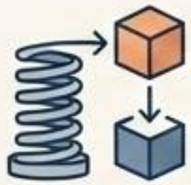
Transparency & auditability

Every agent action is logged, enabling full traceability.



Regulatory alignment

Agent roles can be mapped directly to governed financial functions.



Flexibility

Agents can be added or removed dynamically as requirements evolve.



Intuitive analogy

MAS functions like a digital trading floor, with specialized agents collaborating in real time.

Strategic Impact & Frontier Challenges

- → MAS operationalize the division of labor as a digital-native architecture in finance.
- → They unlock scalable specialization while enhancing robustness and modularity.
- → Applications span front office (trading), middle office (risk), and back office (compliance).
- → The frontier challenge shifts from capability to orchestration—coordination, latency, and governance.

Building Blocks of Multi-Agent Systems (MAS)

Core Components

Collaboration Patterns: Define coordination & task distribution.

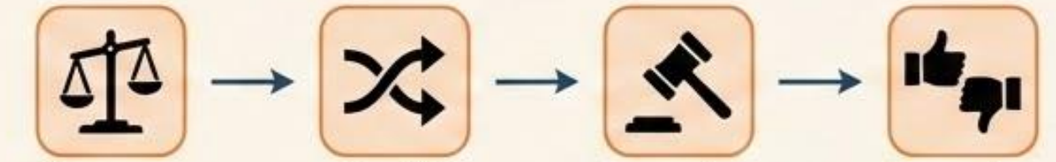
Communication Protocols: Govern information sharing & exchange.

Memory Structures: Determine information retention & recall.

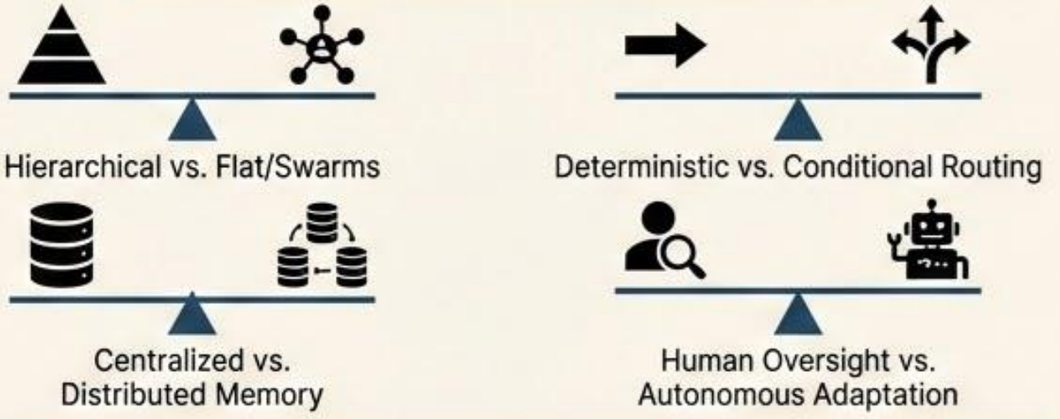
Orchestration Frameworks: Manage & scale agent workflows (e.g., LangGraph, Ray, Kubernetes).

Strategic Considerations & Trade-offs

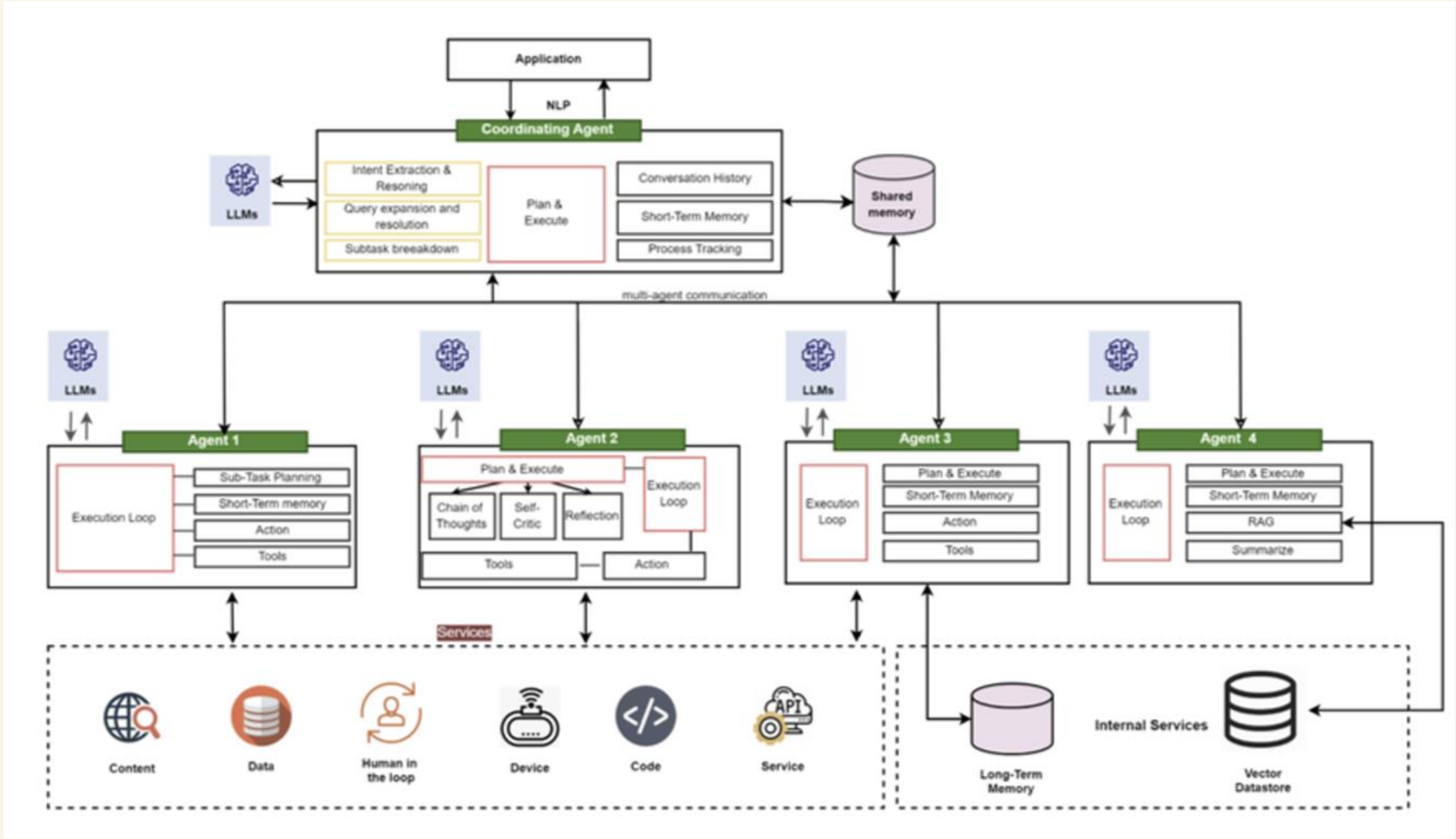
Orchestration in Finance: Why it Fits



Key Design Trade-offs



Multi-Agent Architecture



https://jonturow.substack.com/

Memory in AI Agents: Importance & Components

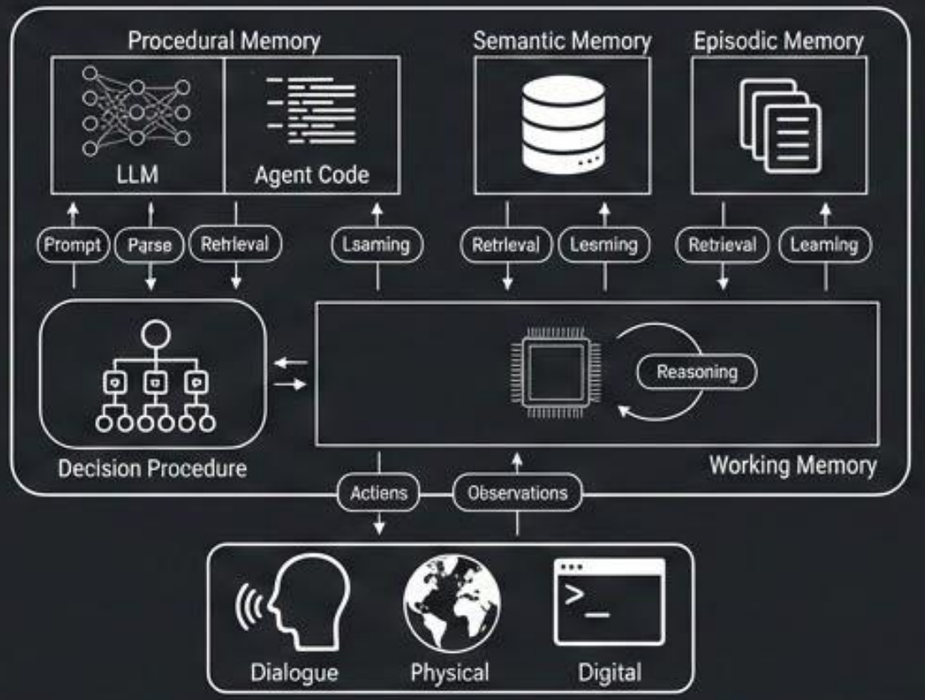
WHY IT MATTERS

Continuity & Context (Avoids statelessness)

Financial & Compliance Tracking (Portfolio, logs)

Bridges Single-Step to Multi-Step Reasoning

MEMORY OVERVIEW DIAGRAM



CORE COMPONENTS & TYPES

Memory Types

Short-term: In-session Context (Chain of reasoning)

Long-term: Cross-session Persistence (Historical knowledge)

Memory Categories

Episodic (Events) → **Semantic** (Facts) → **Procedural** (Skills)

Image source: Cognitive Architectures for Language Agents, https://arxiv.org/abs/2308.02427

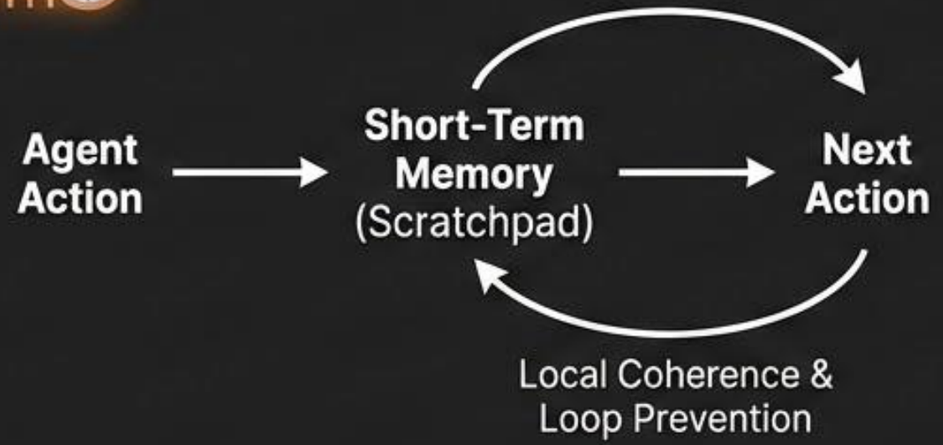
Short-Term vs. Long-Term Memory in AI Agents

Comparing transient in-session context with persistent cross-session knowledge.

Short-Term Memory (Working Memory)



Transient, In-Session Context. Stored in agent state (e.g., LangGraph state object).



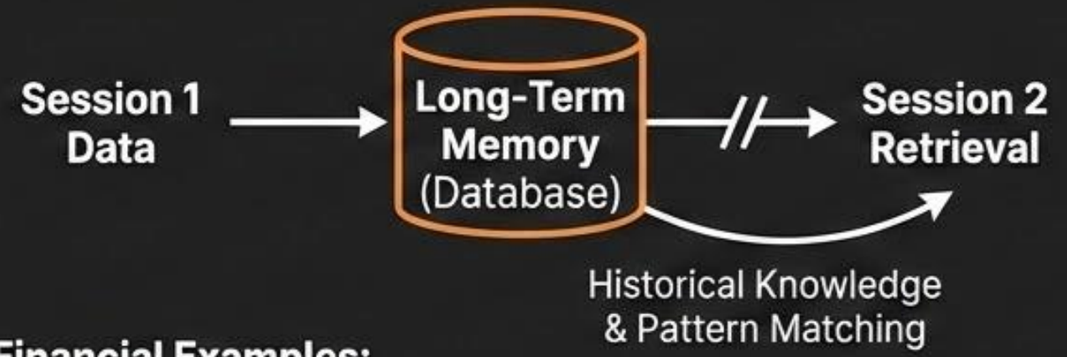
Financial Examples:

- **Execution Agent:** Remembers last trade orders.
- **Risk Agent:** Recalls last exposure calculation.

Long-Term Memory (Persistent Storage)



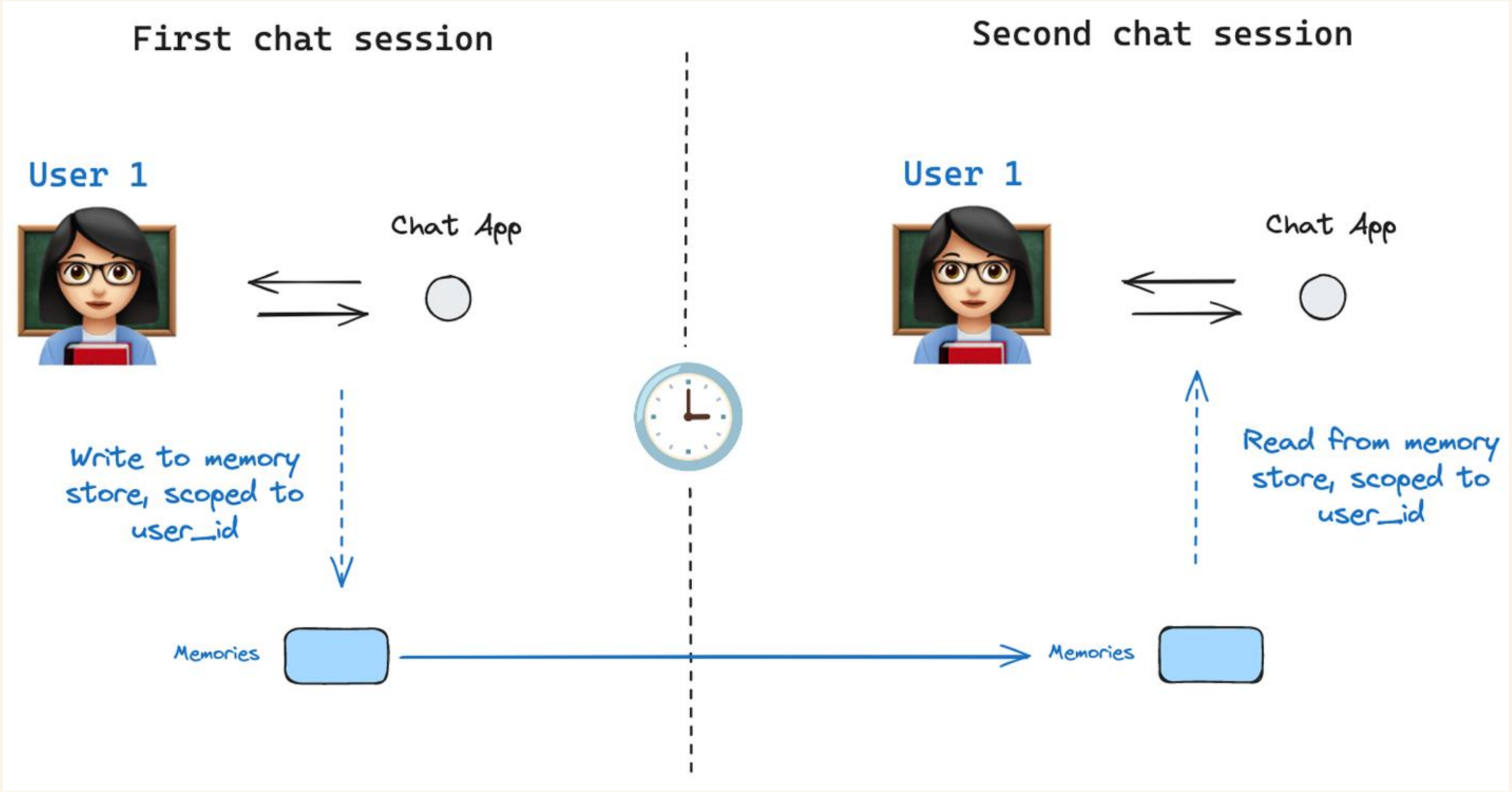
Persistent, Cross-Session Knowledge. Stored externally (e.g., Vector DB, SQL).

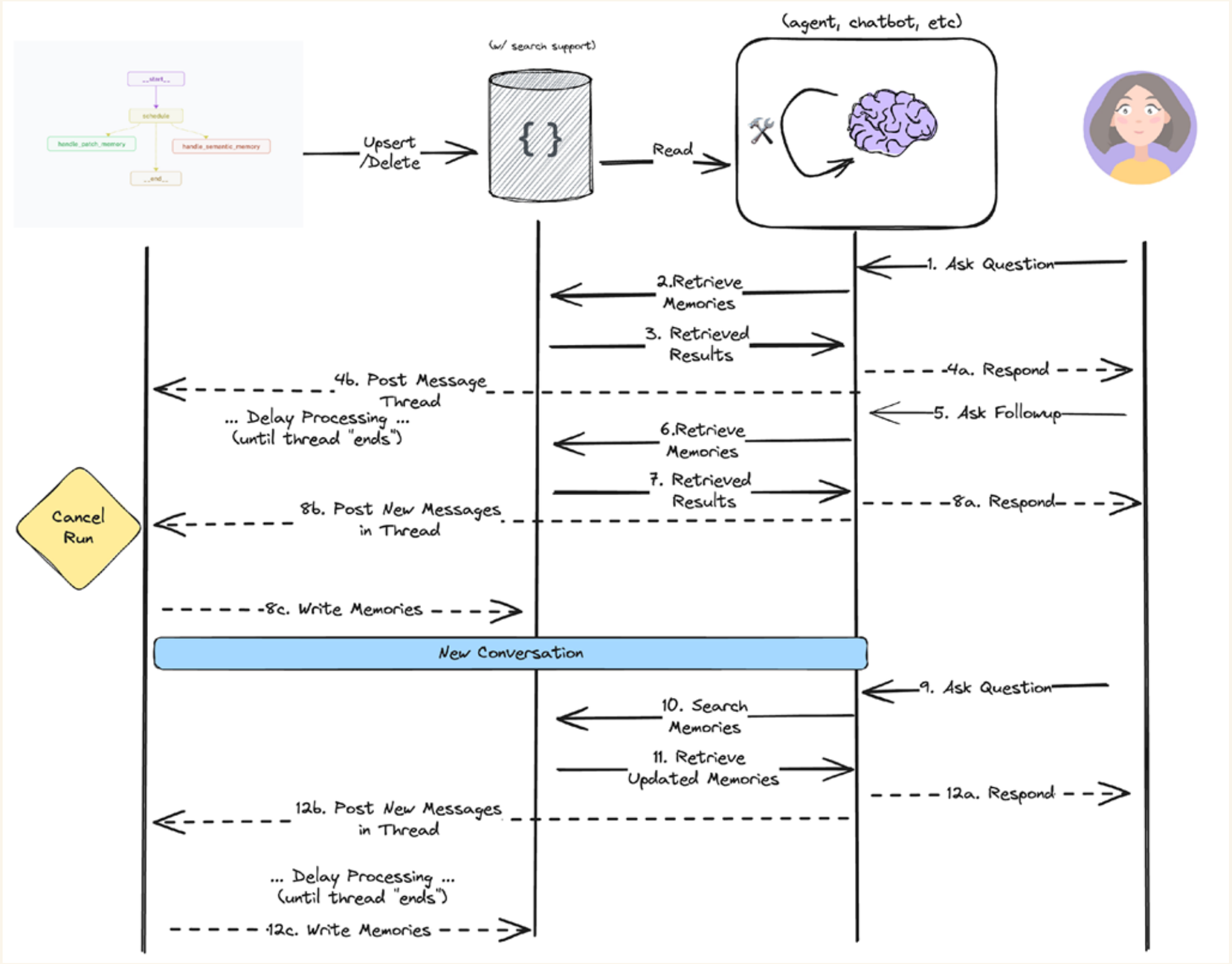


Financial Examples:

- **Portfolio MAS:** Retains allocation history.
- **Compliance MAS:** Retains past regulatory violations.
- **Fraud MAS:** Stores flagged patterns for future matching.

LangGraph Memory Store

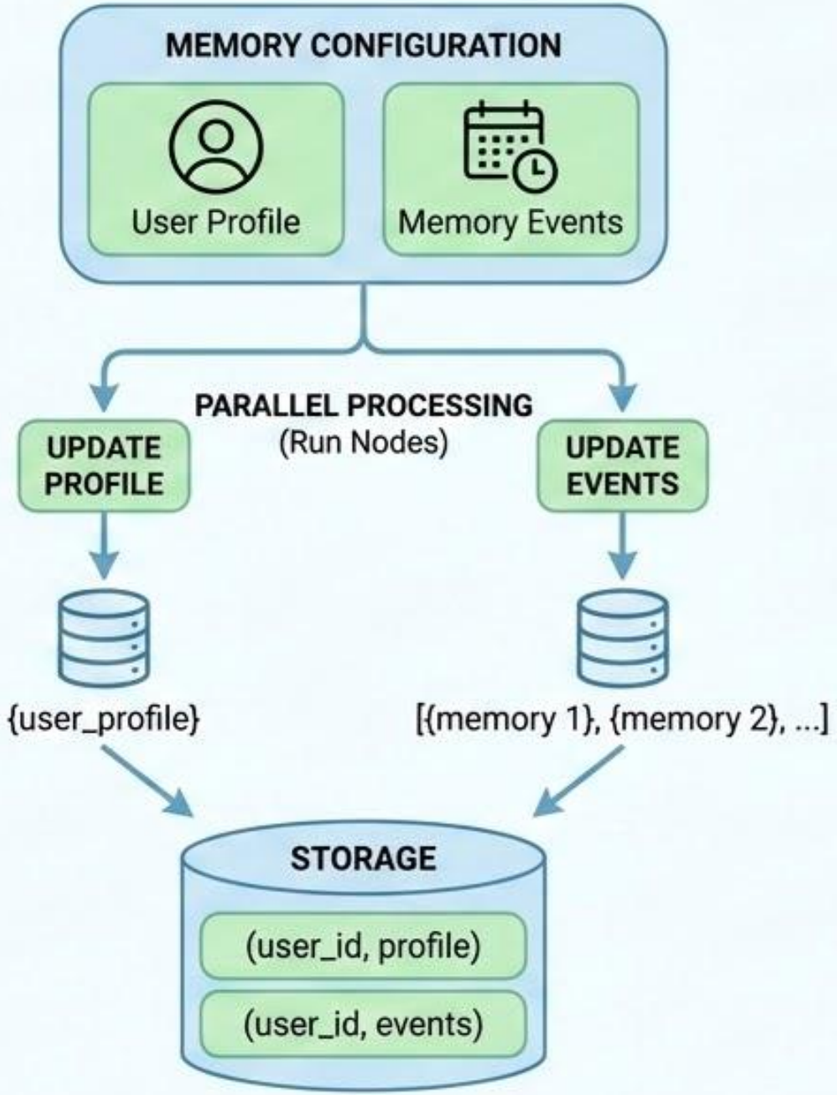




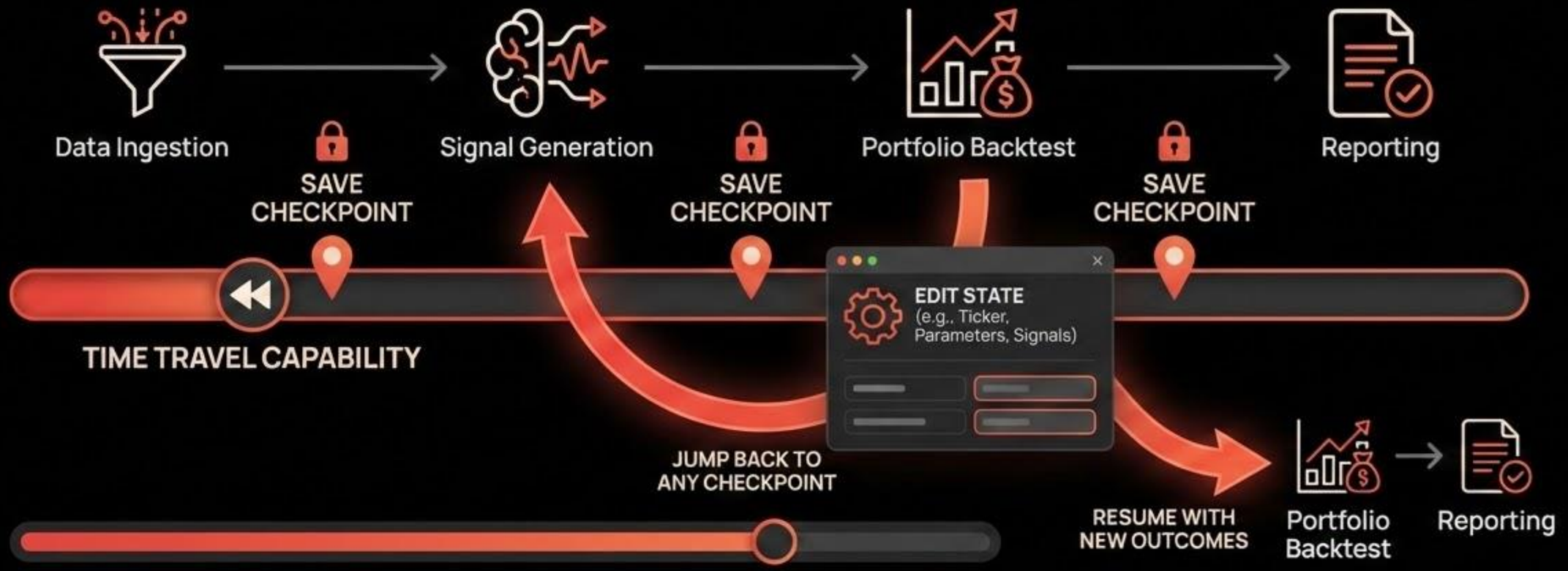
https://github.com/langchain-ai/langgraph-memory/blob/main/example.ipynb

How it works

- When should memories be formed?
- What should each memory contain?
- How should memories be updated?



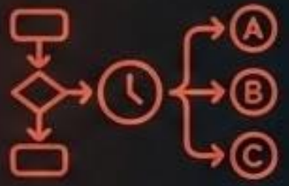
The Core Idea: Financial Workflow with ‘Time Travel’



- **Save** checkpoints automatically during execution
- **Edit state** at checkpoint (e.g., adjust parameters)
- **Jump back** to any prior checkpoint
- **Resume** execution to generate alternative outcomes

Why It Matters in Finance

1. Efficient Scenario Testing & 'What-Ifs'



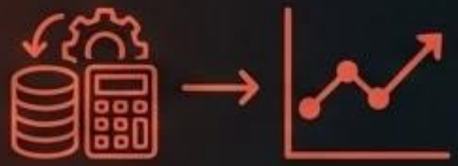
Avoid wasteful reruns. Test variations (tickers, parameters) by branching from a checkpoint instead of restarting the entire pipeline.

2. Minimized Data Overhead



Fetch market data once. Replay downstream steps multiple times from a checkpoint, bypassing slow or rate-limited data pulls.

3. Agile Cost-Aware Backtesting



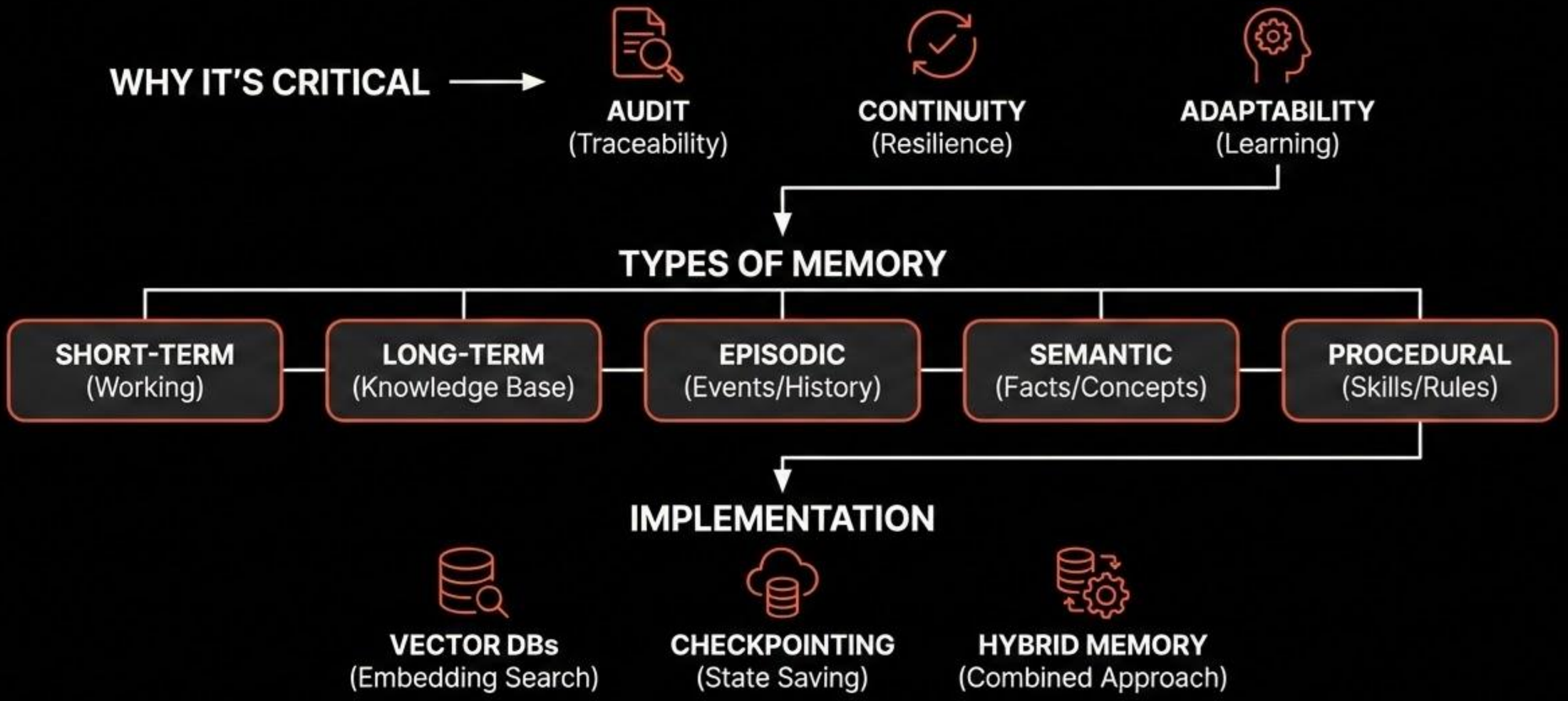
Apply complex cost models (slippage, fees) quickly. Checkpoint before the cost step and iterate on assumptions without re-executing.

4. Robust Production Workflows



Pause, inspect state, and adjust logic on-the-fly. Resume execution from a checkpoint for faster debugging and consistent results.

MAS MEMORY: CRITICAL FOR FINANCE



Why Collaboration Patterns Matter

Structured Coordination (MAS)

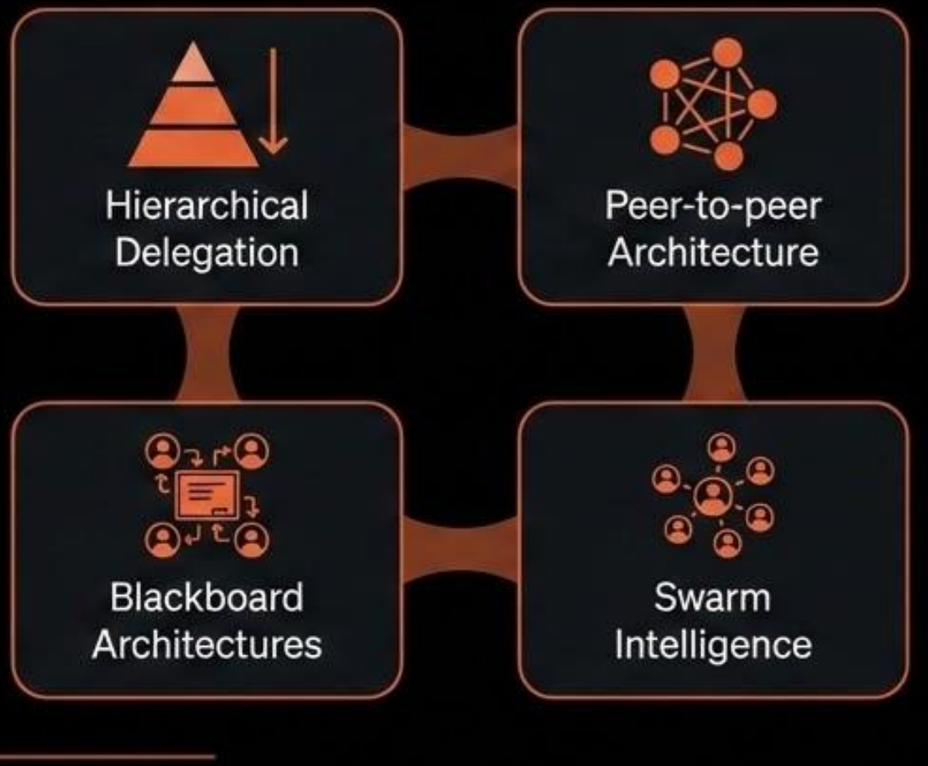
Defining Coordination

Collaboration patterns define:

- How work is divided
- How decisions are reconciled
- How failures are contained

Different financial problems → different collaboration patterns.

The Four Core Patterns



Hierarchical Delegation

Core Concept

- One supervisor agent delegates tasks to specialized sub-agents.
- Supervisor decides sequencing, error handling, and integration of outputs.

Finance Analogy

- Portfolio manager (Supervisor) → team of analyst, trader, risk officer.
- Supervisor agent (portfolio strategy) delegates:
  - Allocation
  - Execution
  - Risk monitoring
  - Compliance



Pros & Cons (Finance Lens)

Pros	Cons
Clear accountability	Bottleneck at supervisor
Simplified coordination	Risk of single point of failure
Fits regulated environments	Less adaptive to unexpected events

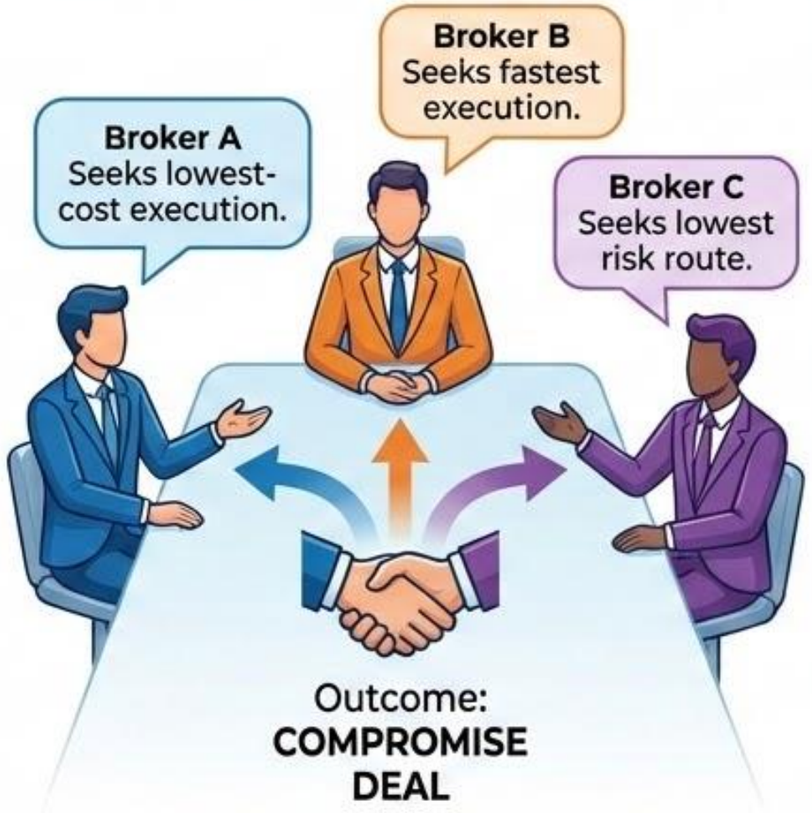
Peer-to-Peer (P2P) Multi-Agent Systems: Decentralized Coordination

CORE CONCEPT: EQUALS IN COLLABORATION

- Agents **act as equals**, no central controller.
- Each has **local objectives**; they **negotiate** & **coordinate** directly.



FINANCE ANALOGY: LIQUIDITY NEGOTIATION



Real-time negotiation between brokers to find a mutually acceptable trade.

PROS & CONS: SYSTEM TRADEOFFS

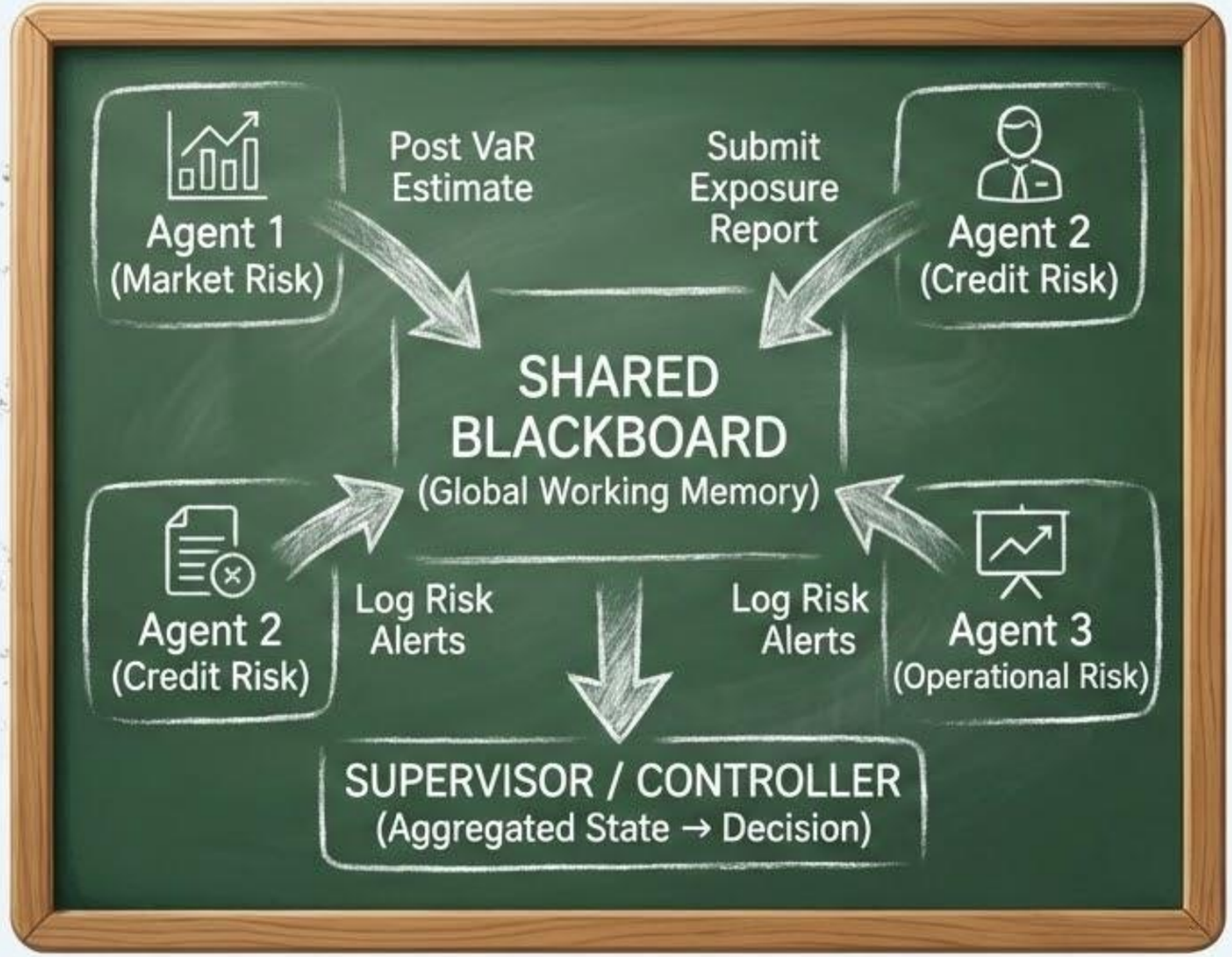
PROS (Advantages)

- ✓ **Resilient** (No single bottleneck/failure point)
- ✓ **Highly Adaptive** to changing market conditions

CONS (Challenges)

- ✗ **Complex coordination overhead** (chatty)
- ✗ **Harder to regulate**, audit, & monitor
- ✗ **Risk of collusion** between agents

Blackboard Architectures: A Collaborative Model



Core Concept: Independent agents collaboratively solve problems by posting partial solutions to a shared workspace, which is monitored and acted upon by a supervisor.

System Characteristics (Pros & Cons)	
✅ **PROS**	❌ **CONS**
✅ Transparent, auditable workflow	❌ Central Point of Failure (The Blackboard)
✅ Scalable - Agents build on each other's results	❌ Potential for bottlenecks & inconsistency
✅ Flexible integration of diverse expertise	❌ Complex synchronization & communication overhead

Swarm Intelligence

Core Concept & Inspiration



- Many simple agents, local interactions → emergent global behavior.
- Inspired by biological swarms (ants, bees, birds).

Finance Analogy & Process

Simulating Trading Strategies:

📈 Momentum traders follow price trend.

🔍 Value traders seek undervalued assets.

⚖️ Arbitrage traders exploit mispricings.



Emergent outcome: collective market dynamics.

Summary

Pattern	Finance Example	Advantages	Limitations
Hierarchical	Portfolio manager MAS	Clear ownership, strong governance, well-defined structure	Can create bottlenecks, slower to adapt to new information
Peer-to-Peer	Cross-venue liquidity	Flexible and resilient, adapts quickly to changing conditions	Coordination complexity, potential for misalignment/collusion
Blackboard	Risk aggregation	High transparency, easy to audit and trace decisions	Risk of information overload, synchronization challenges
Swarm	Market simulation	Highly scalable, enables emergent intelligence	Hard to control, low interpretability / lack of transparency

Agentic AI Patterns:

Pattern	Description	Finance Use Case
Reflective Agent	Continuously evaluates and refines its own outputs to improve decision quality.	Portfolio optimization agent that stress-tests allocations and adjusts strategy pre-trade
Task-Oriented Agent	Executes clearly defined tasks with precision and reliability.	Corporate actions engine handling dividends, splits, and mergers with full accuracy
Hierarchical Agent	Organized in layered control structures with centralized oversight	Enterprise risk system where a top-level controller oversees VaR, stress testing, liquidity
Coordinating Agent	Orchestrates collaboration and task delegation across multiple agents.	Trading orchestrator routing orders, monitoring execution quality, enforcing compliance
Distributed Ecosystem	Operates across decentralized agents interacting in a shared environment.	DeFi network with liquidity providers, arbitrage bots, and validation agents
Human-in-the-Loop	Combines automated decision-making with human oversight for critical controls	Credit risk agent proposing decisions with final approval from a human officer
Self-Learning Agent	Improves continuously through feedback and historical interaction data.	Trading co-pilot adapting to PM preferences and evolving market conditions
RAG-Based Agent	Enhances decisions by dynamically retrieving and integrating external knowledge.	Due diligence agent pulling filings, research, and ESG data in real time
Planning Agent	Develops and executes multi-step strategies to achieve complex objectives.	Wealth management agent building and rebalancing portfolios based on goals and macro inputs
Context-Aware Agent	Dynamically adjusts behavior based on real-time context and environmental signals.	FX hedging agent adapting during volatility spikes or macro events

How to choose the Right Multi-Agent Pattern

1. Objective Alignment

Optimize for **Speed**

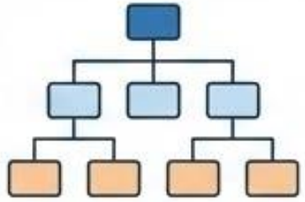
Optimize for **Auditability**

Optimize for **Adaptability**

Goal-Dependent Selection

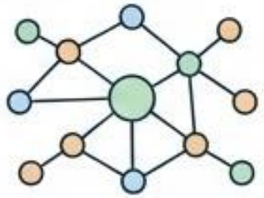
2. Regulatory Constraints

Tightly Regulated



Structured & Auditable
(e.g., Hierarchical, Blackboard)

Exploratory / Innovation



Decentralized Approaches
(e.g., Swarm)

3. System Scale

Small Scale



Few Specialized Agents

Large Scale



Large Network of Lightweight, Distributed Agents

Core Architecture Layer: Multi-agent patterns align with specific financial workflows.

Fundamental Design Trade-offs

Control & Transparency ⟷ **Flexibility & Resilience**

Communication Protocols – How agents exchange information and coordinate actions

- **Message Passing Systems**
- **Shared memory approaches**
- **Event-driven coordination**



Why Communication matters?

Collaboration & Communication

Collaboration patterns determine structure & interactions.

Communication protocols govern information flow.

In Finance: Critical Impacts



Latency & P&L

Speed is a competitive edge or bottleneck. Directly impacts Profit & Loss.



Auditability

Non-negotiable due to regulatory requirements. Ensures transparency.



System Robustness

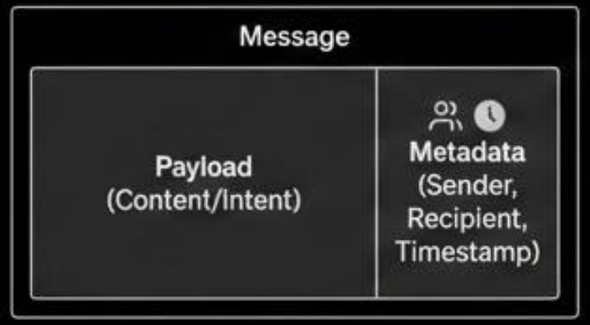
Critical to withstand noise, errors, and adversarial behavior. Ensures reliability.

Message Passing Systems

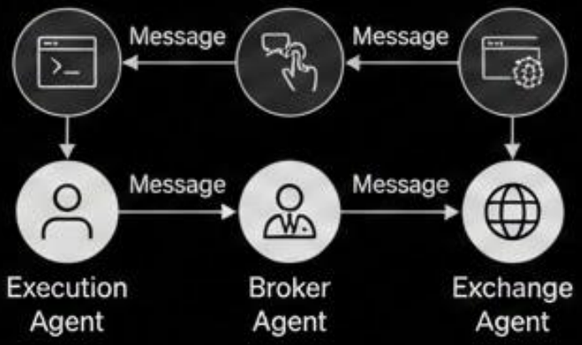
Concept & Structure

- Direct communication between agents via messages.
- Inspired by actor-model systems, distributed computing.

Message Structure:



Finance Analogy: Order Routing



Example Message:

Intent: "buy 100 AAPL @ market" +
State: "Portfolio ID, Compliance Flags"

Pros & Cons

- **Pros (✅)**
  - Explicit and modular.
  - Fine-grained control over communication.
  - Easier to debug (inspect message logs).
- **Cons (❌)**
  - Latency overhead (network-like behavior).
  - Risk of message loss or duplication.
  - Complex at scale.

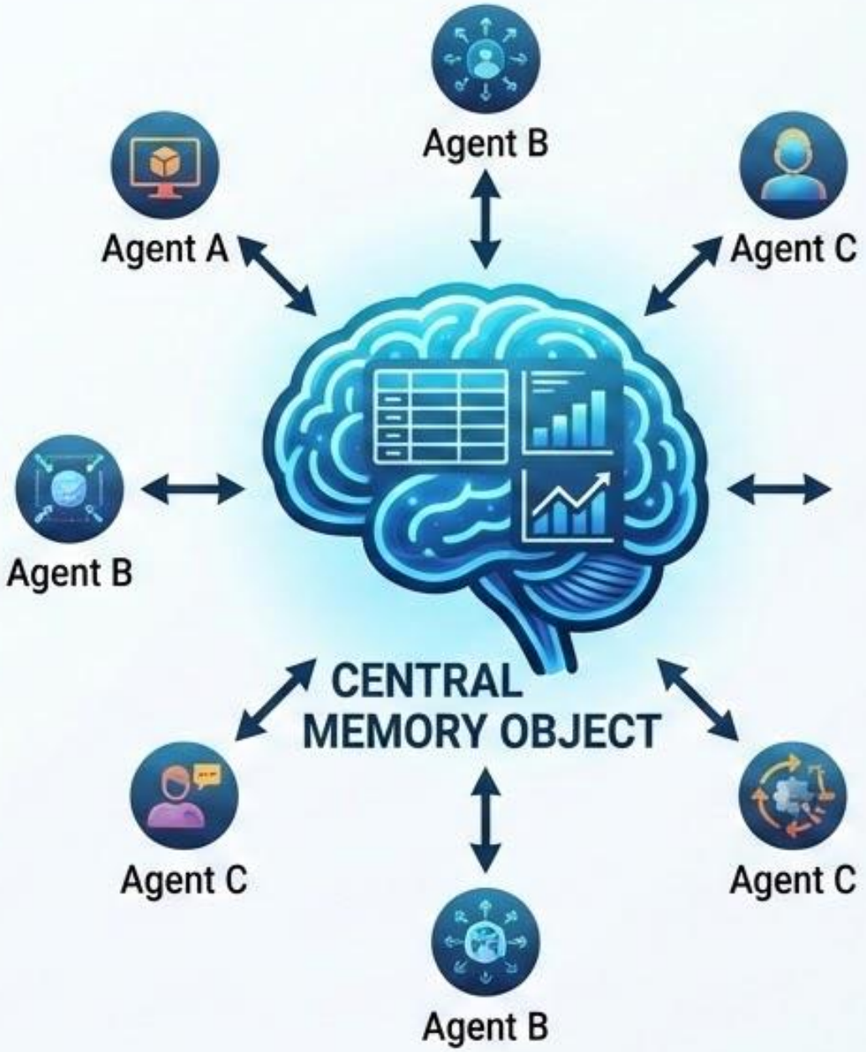
Shared Memory Approaches & Trade-offs

CONCEPT & STRUCTURE

Agents read/write to a central memory object.

Memory = structured state (market snapshot, trade book, compliance status).

Each agent can add, update, or consume from this memory.



PROS & CONS

PROS

- Simple and transparent view of state.
- Natural fit for logging and auditing.
- No need for complex routing.

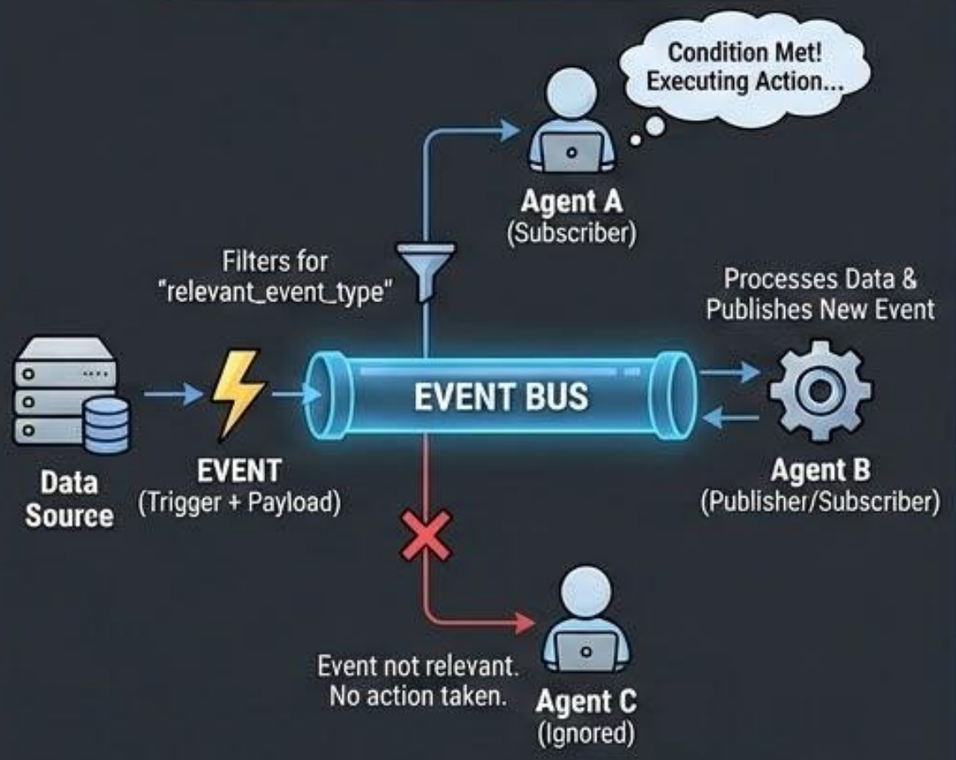
CONS

- Synchronization overhead.
- Race conditions (two agents writing simultaneously).
- Single point of failure.

Event-Driven Coordination in Multi-Agent Systems

Agents react asynchronously to significant changes ("events") broadcast on a shared channel, enhancing efficiency and scalability.

1. CONCEPT & WORKFLOW



- **Decoupled interaction:** Agents don't know about each other directly.
- **Asynchronous:** Agents act only when triggered, no constant polling.
- **Payload carries data** (e.g., new price, threshold breached).

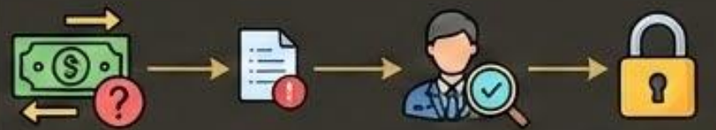
2. FINANCE ANALOGY (Examples)

Stop-Loss Execution



Trigger: Price breach. **Action:** Immediate sell.

Fraud Alert



Trigger: Anomaly pattern. **Action:** Investigation/Alert.

3. PROS & CONS

PROS



Reactive & Efficient: Resources used only when needed.



Reduces Lateney: Near real-time response to changes.



Highly Scalable: Easily add specialized agents.

CONS



Complex Design: Requires robust event schema & handling.



Risk of Failure: Missed or out-of-order events can be problematic.



Difficult Debugging: Tracing distributed event flows is challenging.

COMPARATIVE OVERVIEW: MAS PROTOCOLS IN FINANCE



MESSAGE PASSING

FINANCE EXAMPLE

Execution AGENT → Broker → Exchange

Order routing (Explicit, auditable)

PROS

Explicit, auditable

CONS

Latency overhead



SHARED MEMORY

FINANCE EXAMPLE

Agent A, Agent C ↔ MARKET STATE BUFFER ↔ Agent B, Agent C

Market state buffer for all agents (Transparent, simple)

PROS

Transparent, simple

CONS

Race conditions, central bottleneck



EVENT-DRIVEN

FINANCE EXAMPLE

TRIGGER (e.g., price drop) → EVENT → Stop-loss triggers, Fraud alerts

Stop-loss triggers, fraud alerts (Efficient, scalable)

PROS

Efficient, scalable

CONS

Event complexity, harder debugging

Finance MAS Implementation & Technical Stack

Execution MAS (Broker ↔ Exchange)

Message Passing (Speed & Modularity) → Low-latency Message Queues (e.g., Kafka, ZeroMQ)

Risk MAS (Portfolio Exposures)

Shared Memory (Audit Trails) → Persistent Shared Memory (Databases, Vector/Graph DBs)

Compliance MAS (Suspicious Trades)

Event-Driven (Reactivity) → Event-Driven Triggers

Security & Audit Requirements (Encryption & Logs for ALL Protocols)

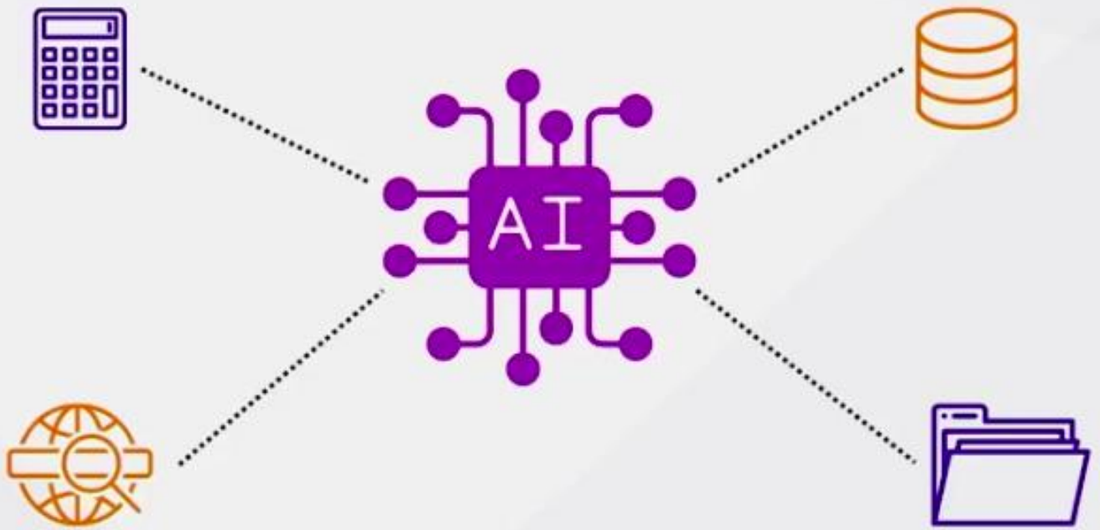
Summary

- **Communication protocols serve as the core infrastructure** of multi-agent systems, enabling coordination and execution.
- **Every protocol involves trade-offs**, requiring careful selection based on system priorities.
- In practice, **financial MAS adopt hybrid approaches**, combining multiple protocols to align with specific objectives.

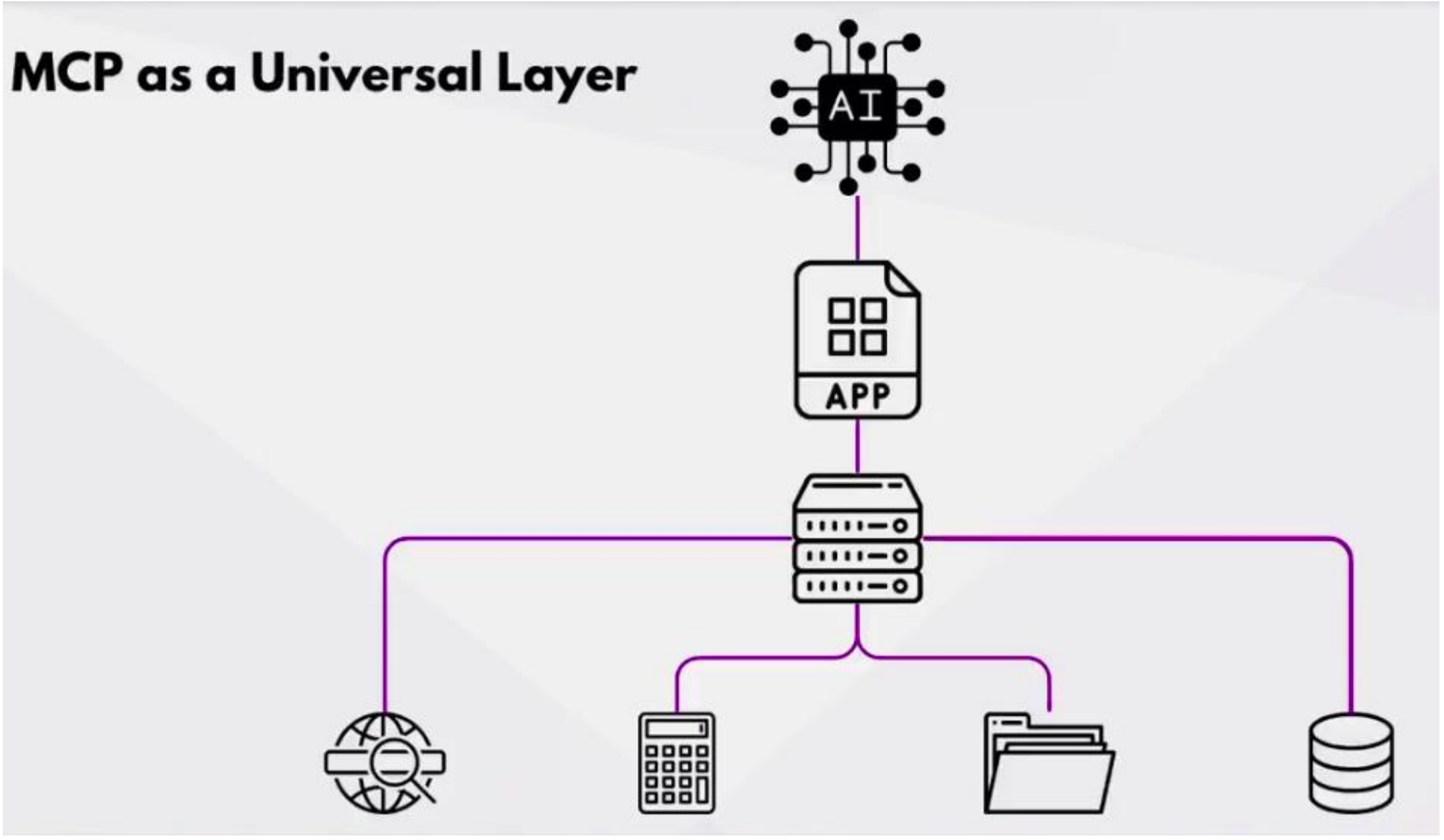
MCP Lab

What is MCP?

An open protocol for connecting AI models to tools, APIs, and external context.

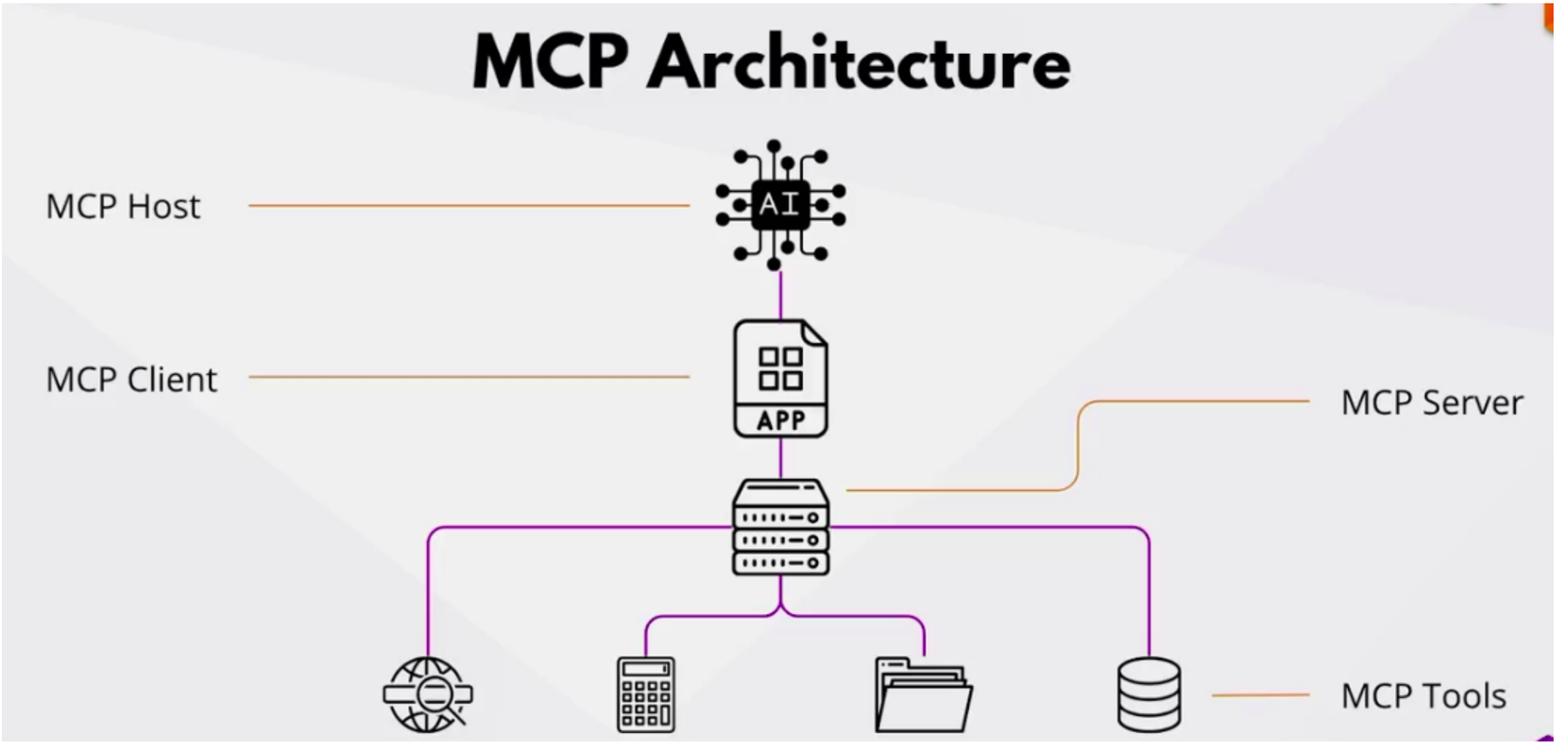


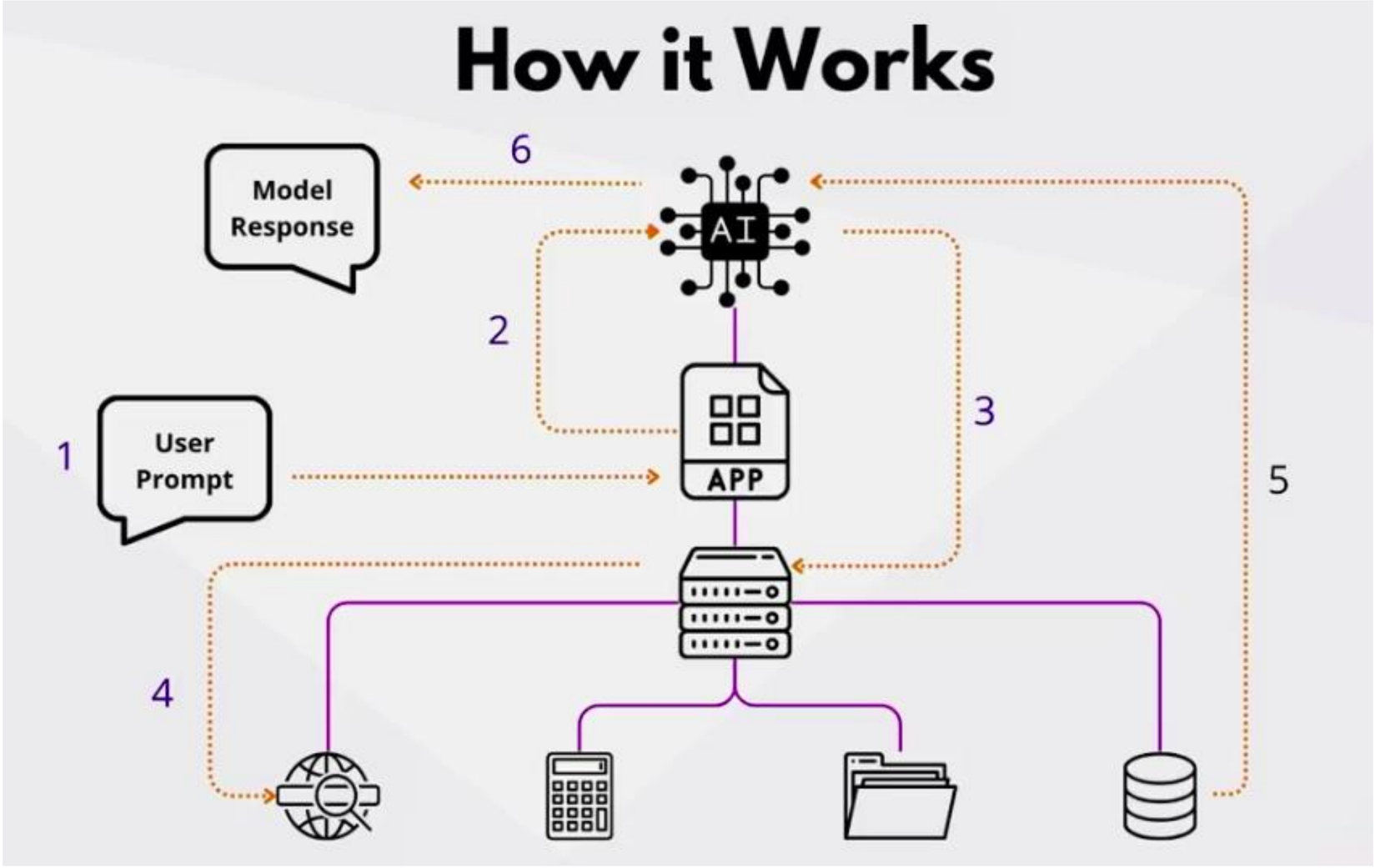
MCP acts as a bridge between models and the outside world.


MCP as a Universal Layer
AI
APP

Benefits of MCP

Benefit	What it Means
Interoperability	Connect across platforms and tools
Consistency	Tools behave the same, regardless of the model
Reusability	Build once, plug in anywhere
Faster Development	Skip boilerplate, focus on logic


MCP Architecture
MCP Host
AI
MCP Client
APP
MCP Server
MCP Tools


How it Works
Model Response
6
AI
2
1
User Prompt
APP
3
5
4

Real World Use Cases

01. Enterprise Tools

Connect your model to internal business systems like CRMs, databases, or ticketing platforms to automate and streamline workflows.

02. Agentic AI

Enable models to decide which tools to use based on user intent, supporting autonomous reasoning and dynamic tool selection.

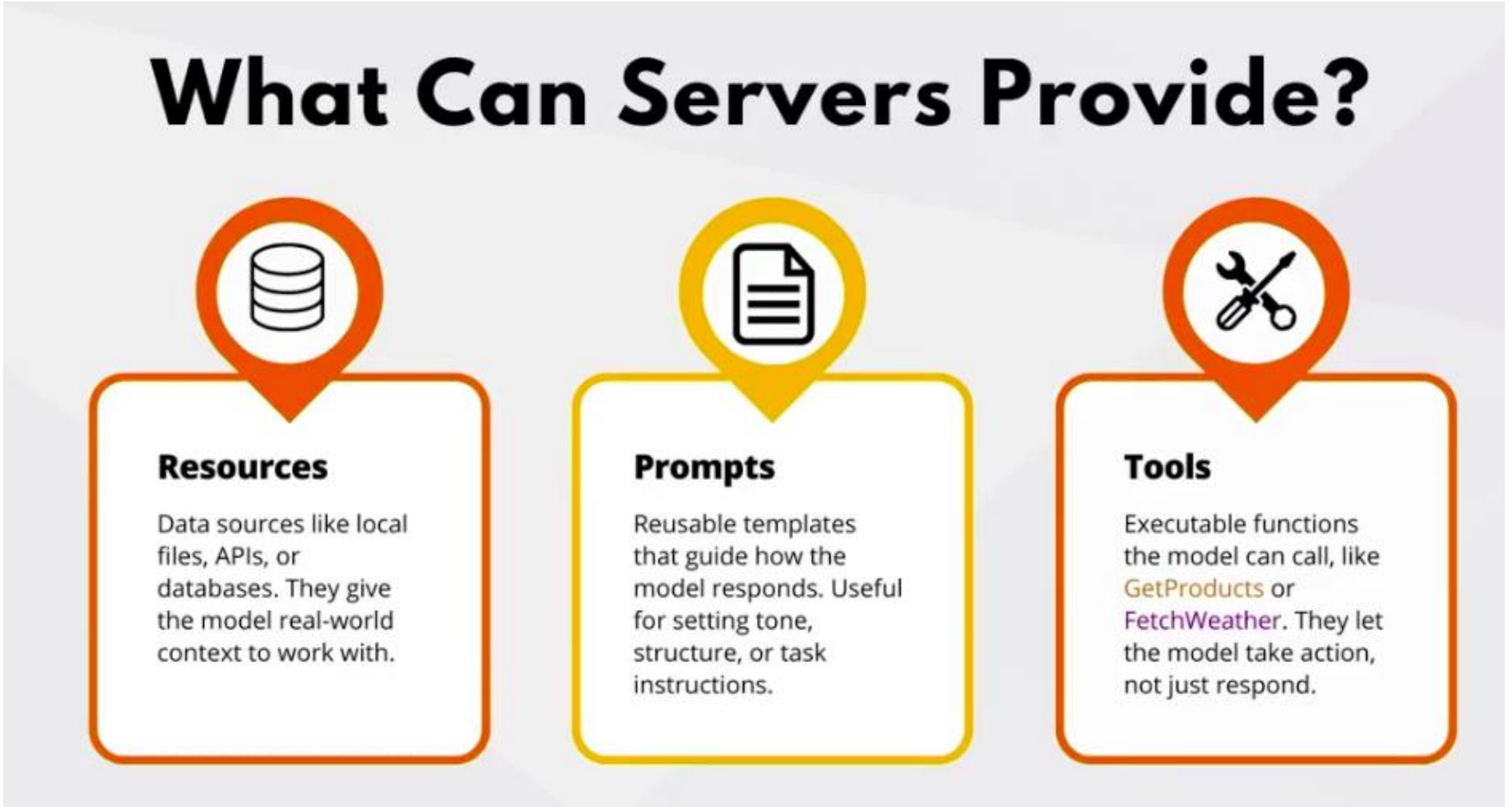


03. Multimodal Apps

Build applications that go beyond text by integrating image generation, audio processing, and other sensory tools.

04. Real-Time Data

Give your model access to live information like stock prices, weather, or breaking news to generate up-to-date and accurate responses.



The main security challenges introduced by MCP include:

Prompt Injection: Malicious instructions hidden in external contexts like emails or web pages that cause unintended actions.
Tool Poisoning: Tampering with the metadata of MCP tools to introduce dangerous behavior.
Token Mismanagement: Issues with token pass-through that can bypass security controls and break trust boundaries.
Excessive Permissions: MCP servers getting too much access to sensitive data, violating the principle of least privilege.
Supply Chain Security: Ensuring the security of models, embeddings, APIs, and context providers integrated into the AI application.

Key Takeaways

- MCP connects AI models to real-world tools and data
- It uses modular architecture with **Hosts, Clients, and Servers**
- Secure, extensible, and language-agnostic

MCP SDK Capabilities

Starter templates are included—no need to start from scratch.

- Build simple MCP clients
- Launch full-featured servers
- Use Tools, Prompts, and Resources

What Makes Up an MCP Server?



Key Takeaways

- MCP works across a wide range of scenarios—from simple automation to complex multi-agent systems.
- It **integrates cleanly** with Azure tools, OpenAI models, and web or desktop environments.
- Reusable components and architectural best practices can **help you move faster**.
- You don’t need a huge project to get started—even **lightweight use cases** can show ROI quickly.

Step 1) Create a project folder

Create a new folder and open it in VS Code:

```
option-mcp-demo/
  server.py
  requirements.txt
  .vscode/
    mcp.json
```

Step 2) Add dependencies

Create **requirements.txt**:

mcp

Step 3) Create the MCP server (Echo + Delta + Gamma)

Step 4) Configure VS Code to launch your MCP server

Create `.vscode/mcp.json`.

macOS / Linux

```json
{
  "servers": {
    "option-mcp": {
      "type": "stdio",
      "command": "${workspaceFolder}/.venv/bin/python",
      "args": ["${workspaceFolder}/server.py"]
    }
  }
}
```

Step 5) Start the MCP server inside VS Code

In VS Code:

1. Open the Command Palette: **Cmd** + **Shift** + **P**
2. Search for "MCP: List Servers"
3. Start **option-mcp**

Once started, VS Code will manage the lifecycle and display logs if needed.